

PA1: DOGCAT

Question 1: Heuristic design and admissibility

Steps

$h(n) = d$, the number of letters in n that don't match the goal.

This lower-bounds the number of moves left. Each move changes only one letter, so at least d moves are needed, and each costs 1. So h never overestimates.

Scrabble

$h(n)$ = the sum of the Scrabble values of the goal letters still missing.

This lower-bounds the cost of putting the goal's letters in place. Each wrong position must eventually get its goal letter, and that move costs exactly that letter's value. Any extra moves only add cost, so h never overestimates.

Frequency

$h(n) = 0$ at the goal. Otherwise $h(n) = (d - 1)(1 + r_{\min}) + 1 + \text{rarity}(\text{goal})$, where $r_{\min}$ is the lowest rarity of any word with the same length.

This lower bounds the cost of the words still to visit. At least d moves are needed. The last move lands on the goal, costing exactly $1 + \text{rarity}(\text{goal})$, and every other move costs at least $1 + r_{\min}$. So h never overestimates. Since rarity is never negative, $h \geq d$, so it isn't the trivial zero heuristic.

Question 2: Does the heuristic actually help?

Word Pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY -> ASK	Steps	5	773	5	16	97.93%
WHY -> ASK	Scrabble	9	765	9	13	98.30%
WHY -> ASK	Frequency	38.86	808	38.86	634	21.53%
BEAR -> DUCK	Steps	4	691	4	6	99.13%
BEAR -> DUCK	Scrabble	11	2981	11	6	99.80%
BEAR -> DUCK	Frequency	38.24	1084	38.24	257	76.29%

$$\text{Reduction} = 100 \times (\text{UCS expansions} - \text{A* expansions}) / \text{UCS expansions}$$

1. Did UCS and A* always return the same optimal cost?

Yes, they matched in all 6 cases. That's expected, since A* with an admissible heuristic always finds the optimal cost.

2. Largest reduction for each pair:

Scrabble for both: 98.30% for WHY -> ASK and 99.80% for BEAR -> DUCK.

3. Least informative heuristic for each pair:

Frequency for both: 21.53% for WHY -> ASK and 76.29% for BEAR -> DUCK.

4. Explanation:

The steps and Scrabble heuristics are very close to the true cost. For BEAR -> DUCK, all 4 letters differ and the best path is exactly 4 moves, so h is exact at every node (all deltas are 0). A* only expanded 6 nodes, while UCS expanded hundreds or thousands. Scrabble does slightly better than steps because its letter costs are higher, so bad moves stand out more.

For WHY -> ASK, only 3 letters differ, but the best path takes 5 moves, so h underestimates a bit and A* explores a few more nodes.

Frequency has the weakest heuristic. It assumes the in between words have the lowest possible rarity, but real paths use words with much higher rarity, so h is far below the true cost (first deltas of -31.59 and -25.96). This makes A* act more like UCS. WHY -> ASK is the worst case because its best path (why -> way -> was -> has -> his -> ais -> ass -> ask) takes 7 moves when only 3 letters differ. BEAR -> DUCK does better because its path is exactly 4 moves for 4 different letters.

Question 3: Reversibility and direction-dependent costs

Action reversibility vs. cost symmetry: A DOGCAT action is always reversible: if you can change W_1 into W_2 , you can change W_2 back into W_1 . But being able to reverse a move doesn't mean the reverse move costs the same, because the cost depends on the direction.

Steps: Yes.

Every move costs 1 no matter which direction it goes. Any path from W_1 to W_2 can be reversed

into a path from W2 to W1 with the same number of moves, so the optimal cost is the same both ways.

Scrabble: No.

The cost depends on the letter being inserted, and that changes with direction. Going forward inserts one letter, and going backward inserts the letter that was there before.

Counterexample: WHY $\rightarrow$ ASK costs 9, but ASK $\rightarrow$ WHY costs 14.

- WHY $\rightarrow$ ASK: why $\rightarrow$ wha $\rightarrow$ aha $\rightarrow$ ahs $\rightarrow$ ass $\rightarrow$ ask inserts a, a, s, s, k = $1 + 1 + 1 + 1 + 5 = 9$
- ASK $\rightarrow$ WHY: ask $\rightarrow$ ass $\rightarrow$ ahs $\rightarrow$ aha $\rightarrow$ wha $\rightarrow$ why inserts s, h, a, w, y = $1 + 4 + 1 + 4 + 4 = 14$

Frequency: No.

The cost is paid for each word you enter, and the start word is free. Going forward you pay for the goal word but not the start word; going backward it's the opposite.

Counterexample: WHY $\rightarrow$ ASK costs 38.86 but ASK $\rightarrow$ WHY costs 37.76. Both use the same words in reverse order (why, way, was, has, his, ais, ass, ask), but one direction pays for "ask" and the other pays for "why". The difference of 1.10 is $\text{rarity}(\text{ask}) - \text{rarity}(\text{why})$.

Question 4: LLM heuristic audit

1. Model and prompt

Model: Claude Opus 5.5

Prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length, three or four

letters, and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By "more informative," we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Proposed heuristic: The model's idea was to charge the real cost of the goal word for the last move, and the cheapest possible cost for every other move.

2. Formula

$d = h_0(n)$, $r_g = \text{rarity of the goal}$, $r_{\min} = \text{lowest rarity of any other word the same length}$.

$h_1(n) = 0$ if n is the goal

$h_1(n) = (1 + r_g) + (d - 1)(1 + r_{\min})$ otherwise

3. Is it admissible? Yes.

You need at least d moves to reach the goal, since each move changes one letter. The last move lands on the goal and costs exactly $1 + r_g$. Every other move costs at least $1 + r_{\min}$. So the real cost is always at least h_1 , which means it never overestimates. Since rarity is never negative, h_1 is always at least h_0 .

The model made one small mistake. It said h_1 is always bigger than h_0 at non-goal states if $r_g > 0$ or $r_{\min} > 0$. But if $r_g = 0$, a word one letter away from the goal gets $h_1 = 1$, the same as h_0 . So it's only bigger there when $r_g > 0$. This doesn't affect admissibility.

4. Repair

No fix was needed for the heuristic, only for that one wrong claim.

5. Test results (frequency cost)

Word pair	h0 cost	h0 expanded	h1 cost	h1 expanded	Reduction
WHY -> ASK	38.86	755	38.86	634	16.03%
BEAR -> DUCK	38.24	720	38.24	257	64.31%

Both found the same optimal cost, and h1 expanded fewer nodes both times, so h1 is worth using. It helped more on BEAR -> DUCK because that path is exactly 4 moves for 4 different letters. WHY -> ASK takes 7 moves for only 3 different letters, so h1 still underestimates a lot. Passing these tests doesn't prove h1 is admissible by itself; the proof in part 3 does.

Generative AI use

I used Claude Opus 5.5 for setup help like installing numpy and for Question 4. I verified everything by running the provided tests and checking the results myself.